When Software Fabricates Assurance: Seven Design Principles for Trustworthy Governance, Risk and Compliance Systems

Name of the Track: Track 1 – Information Systems: Theory and Practice (Digital Ethics and Policy: Regulation and Compliance)

Type of Submission: Ongoing research (design science research; artifact and technical evaluation completed, naturalistic evaluation pending)

Abstract

1. Purpose

Boards, auditors and regulators rely on governance, risk and compliance (GRC) information systems for evidence that risks are identified, rated, approved and controlled. Research on these systems has examined their adoption and value. We found no study asking whether the evidence they present can itself be trusted. A code and runtime audit of an existing risk management application found that the software invented risk ratings, discarded approvals, accepted unauthenticated authority and reported failed database writes as successes. This study asks which design failures compromise the integrity of assurance evidence. It also asks which design principles prevent them, and how well an instantiation of those principles performs against the original system.

2. Design/Methodology/Approach

This is a design science study. We read the code of an existing risk management application, ran it, and recorded seven distinct ways in which it failed. Each failure was traced to theory that already explains it, from integrity models, fail-safe defaults and complete mediation to separation of duties, tamper-evident logging and the critique of audit as ritual, and a design principle was drawn from that theory. We built a new system, ERMA-GRC, on these principles. The old and new systems then faced 23 scenarios over the same stored data, under a protocol time-stamped in the project repository beforehand. Eighteen scenarios come from the audit, four go beyond it, and one was meant to fail. All deviations are reported.

3. Findings

Out of the 18 possible cases resulting from the audit test, the old system blocked the failure of security in 4 of these. On the other hand, ERMA-GRC blocked all 18 of the possibilities. ERMA-GRC blocked writes outside a session, blocking a person from approving his/her record, stamping the audit lines for signed-in users and not the names that are entered, detecting an edit of an entry and reporting unavailability of the database as failure. In addition to these audit test conditions, ERMA-GRC blocked 3 out of the 4 failures possible, where in two cases it was two simultaneous approvals for the same record. Storage results were collected from the database simulator, rather than the system itself. There were 2 scenarios when the new design did not block the failure, and these are mentioned below. Although one risk indicator displayed the right state on the screen, a client can connect to the server and set any state that he/she wishes. Furthermore, a person having access to the audit key can modify the audit trail offline, without detection after server restart. The principles also rely on each other since the reliability of audit depends on separation of duties.

4. Originality/Value

We treat the integrity of assurance evidence as something a GRC system's design can protect or destroy, rather than as an organisational outcome. This paper provides a taxonomy of failures observed in a working system, seven principles together with two testable propositions towards a design theory, and a scenario protocol that others can rerun. They can use the principles as an action list. The two issues in the design give rise to the following three exercises: the anchoring of the audit trail outside of the system; the computation of all metrics in the server, and testing of the principles using practitioners.

Keywords

assurance integrity; GRC information systems; design science research; enterprise risk management; tamper-proof audit trail; separation of duties